

CyberQuant: The Simple Guide to Cyber Risk & Python Backend

Learn Everything From Scratch in Super Plain English (Zero Budget & Free Tools)

A friendly, step-by-step masterclass explaining what the project is, what every tool does, and how you will build the Python backend with zero prior confusion.

Lesson 1: What is This Project? (A Simple Story to Understand Everything)

Hello! Think of me as your personal teacher. Let's start from absolute zero with a simple story.

The Story of the Jewellery Shop:

Imagine your friend opens a giant **Jewellery Shop** in the city. The shop has diamond necklaces worth **₹5 Crores** in the main vault, silver rings worth **₹50,000** in the showcase, and old bills in a storeroom.

One day, a security guard runs to the Shop Owner and screams:

'ALERT! Window #4 has a Threat Level of 9.8 RED!'

The Owner gets confused and asks:

'Wait! Is that window near the diamond vault or the old bills? How much money will I lose if someone breaks in? And should I buy an iron grill (₹10,000) or a laser alarm (₹1 Lakh)?'

The guard has no answer because he only knows how to yell 'Alert!'.

This is EXACTLY what happens in big companies today!

Companies have hundreds of computers (servers) with customer bank details, payment passwords, and employee data. Today's security tools only give scary numbers like 'Severity 9.8'. But company bosses (CEOs) don't speak computer code—they speak **Rupees (₹)**!

What Our Project (CyberQuant) Does:

CyberQuant acts as the **Smart Financial Translator**. It looks at all the broken windows (vulnerabilities), checks what valuable stuff is inside that specific room, calculates the potential loss in **Rupees (₹)**, and tells the boss:

- 'Leaving this window open on your Customer Database means an Expected Annual Loss of **₹18.5 Lakhs**.'
- 'If you spend **₹1.2 Lakhs** to fix the lock today, you will protect **₹9.7 Lakhs** in financial damage!'
- 'You have a **₹1 Crore** budget? Here is the exact combination of 4 locks to buy for maximum safety per rupee.'

Lesson 2: Cybersecurity Words Translated into Plain Everyday English

Fancy Word	Everyday Example (Like in Real Life)	What It Means in Our Project
Asset	Your smartphone, bicycle, or gold jewellery.	Any valuable computer owned by the company (Customer Database worth ₹5 Cr , Payment Server, Employee Laptops).
Vulnerability (CVE)	A broken window latch or a rusty door lock.	A bug or weakness in computer software that bad hackers can sneak through.
Threat	A burglar or pickpocket walking around.	The hacker or ransomware gang trying to attack the company.
Security Control	A CCTV camera, strong padlock, or OTP on your phone.	A protective tool (Multi-Factor Authentication / OTP, Antivirus / EDR, Firewall).
Likelihood (P)	Chance of rain today (e.g. 20%).	The mathematical probability (from 0% to 100%) that an attack will succeed this year.
Expected Annual Loss (EAL)	Probability × Value Lost. 10% chance of losing a ₹10,000 phone = ₹1,000 expected loss/year.	FAIR Framework formula: 20% chance of a ₹1 Crore hack = ₹20 Lakhs Expected Loss/year.
ROSI (Return on Investment)	If you spend ₹10 on a lock to save ₹50 , you made a great deal!	(Money Saved - Cost of Lock) / Cost of Lock. Shows bosses if a security purchase is worth the money.
Knapsack Optimizer	You have ₹500 in your pocket at a snack store. How do you buy the tastiest combo without crossing ₹500 ?	A smart math algorithm that takes your budget (like ₹1 Crore) and picks the best locks that save the most money.

Lesson 3: Every Technology We Use (Explained with Simple Analogies)

Good news: We have ₹0 budget, so everything we use is **100% free and open-source**. Here is what every piece of technology actually does, explained like a restaurant:

Technology	What is it? (Simple Analogy)	Why do we need it in CyberQuant?
Python Core Language	The Master Chef's Language. A friendly programming language that reads almost like plain English.	We write all our calculations, math equations, and data logic in Python because it's super easy to learn and has great math tools.
FastAPI Backend Web Server	The High-Speed Waiter in a Restaurant. The website customer cannot go inside the kitchen. The waiter takes orders and brings food back.	When the user clicks 'Calculate Risk' on the website, FastAPI takes that request, tells Python to do the math, and brings the answer back to the website in milliseconds.
Uvicorn Server Runner	The Waiter's Super-Fast Shoes. A small engine that keeps FastAPI running locally on your laptop.	Runs your Python backend so it listens at <code>http://127.0.0.1:8000</code> .
Pydantic Data Checker	The Security Guard at the Door. Checks tickets to make sure nobody brings invalid items inside.	If someone enters text like 'apple' into the budget box instead of a number, Pydantic stops it so your server never crashes.
SQLite / SQLAlchemy Database Storage	The Digital Notebook & Filing Cabinet. A safe place to write down and remember information.	Stores our 150 fake computers (servers), their vulnerability scores, and history for free on your laptop without paying for cloud DBs.
Pandas & NumPy Math Engine	The Turbo Scientific Calculator. Can add, multiply, and sort 1,000 numbers in 1 millisecond.	Takes our list of 150 servers and calculates the total loss in Rupees across the entire company in a split second.
HTML, CSS & JS Frontend Website	The Beautiful Showroom. • HTML = Walls & buttons • CSS = Colors & dark theme • JS = Light switches (clicks)	Creates the visual web screen where users see the big risk numbers, drag the budget slider, and click buttons.
Chart.js Graph Painter	The Graphic Artist. Draws colorful circular dials, pie charts, and line graphs.	Takes the raw numbers from your Python backend and draws beautiful glowing risk trend charts on the screen.

Lesson 4: How the Whole System Works Step-by-Step

- **Step 1 (The Fake World):** We generate a synthetic enterprise with 150 servers (like Customer DB worth ₹5 Cr, Test Server worth ₹10k) and 100 vulnerabilities.
- **Step 2 (The Math Calculation):** For each server, our Python code calculates: $Expected\ Loss\ (EAL) = Attack\ Probability\ (\%) \times Asset\ Value\ (₹)$.
- **Step 3 (The Waiter Delivers):** When someone opens the web browser, FastAPI sends these numbers to the webpage in clean JSON format.
- **Step 4 (The Artist Draws):** Chart.js on the webpage turns the numbers into glowing dials and trend graphs.
- **Step 5 (The Budget Slider):** The user slides the budget to ₹1 Crore. Your Python Knapsack algorithm picks the best 4 locks to buy in 0.02 seconds!

Lesson 5: Your Personal Masterclass as the Python Backend Developer

Now let's talk about YOUR role! You are the Backend Developer. You are the architect building the central engine room in Python. Here is everything you need to know and write, explained line-by-line:

1. What Are You Building in Simple Terms?

You will write 3 Python files in a folder called `cyberquant_backend`:

- **main.py:** The FastAPI web server that talks to the web browser.
- **risk_engine.py:** The math file that calculates Expected Annual Loss (EAL) in Rupees.
- **optimizer.py:** The smart shopping algorithm (Knapsack) that picks the best security tools for any budget.

2. Installing Your Free Tools (Only 1 Command)

Open your terminal (PowerShell or Command Prompt) and type:

pip install fastapi uvicorn pydantic pandas scipy

That's it! All your backend tools are installed and ready.

3. Full Code Blueprint #1: The Web Server (main.py)

Here is your actual FastAPI server code. Look how simple it is:

```
# 1. Import the free tools
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

# 2. Create the web app
app = FastAPI(title='CyberQuant Backend API')

# 3. Allow web browsers to talk to this Python server (CORS)
app.add_middleware(CORSMiddleware, allow_origins=['*'], allow_methods=['*'], allow_headers=['*'])

# 4. When browser asks for '/api/dashboard', send this answer:
@app.get('/api/dashboard')
def get_dashboard_data():
    return {
        'enterprise_risk_score': 72,
        'expected_annual_loss_inr': 24000000, # █2.40 Crores
        'top_risk_asset': 'Customer Database',
        'potential_risk_reduction_inr': 11000000, # █1.10 Crores
        'monitored_servers': 150
    }
```

4. Full Code Blueprint #2: The Risk Calculation Math (risk_engine.py)

```
def calculate_server_risk(server_value_inr, vulnerability_score, is_internet_facing, mfa_enabled):
    # Step 1: Calculate attack chance from 0% to 100%
    attack_chance = (vulnerability_score / 10.0) * 0.4 # Flaws increase risk
    if is_internet_facing:
        attack_chance += 0.35 # Internet exposure increases risk
    if mfa_enabled:
        attack_chance -= 0.45 # Strong OTP lock reduces risk!

    final_chance = max(0.02, min(0.95, attack_chance)) # Keep between 2% and 95%

    # Step 2: FAIR Formula: EAL = Chance × Value in Rupees
    expected_loss_inr = server_value_inr * final_chance
    return {'chance_percent': round(final_chance * 100, 1), 'loss_inr': round(expected_loss_inr, 2)}
```

Lesson 5 (Continued): The Budget Optimizer & How to Test Everything

5. Full Code Blueprint #3: The Budget Optimizer (optimizer.py)

This algorithm takes any budget (e.g. ₹1 Crore) and picks the best combination of security tools:

```
class BudgetRequest(BaseModel):
    budget_inr: float # e.g. 10000000 for ₹1 Crore

@app.post('/api/investment/optimize')
def optimize_budget(req: BudgetRequest):
    tools_menu = [
        {'name': 'Critical Patching', 'cost': 1000000, 'risk_saved': 2500000},
        {'name': 'MFA Password Lock', 'cost': 2000000, 'risk_saved': 3500000},
        {'name': 'Network Wall', 'cost': 4000000, 'risk_saved': 6000000},
        {'name': 'EDR Antivirus', 'cost': 3000000, 'risk_saved': 4200000},
        {'name': 'Cloud Shield', 'cost': 1500000, 'risk_saved': 1800000}
    ]
    # Sort by best value: (Risk Saved ÷ Cost)
    sorted_tools = sorted(tools_menu, key=lambda x: x['risk_saved']/x['cost'], reverse=True)
    bought, spent, total_saved = [], 0, 0
    for tool in sorted_tools:
        if spent + tool['cost'] <= req.budget_inr:
            bought.append(tool)
            spent += tool['cost']
            total_saved += tool['risk_saved']
    rosi_percent = ((total_saved - spent) / spent) * 100 if spent > 0 else 0
    return {'recommended_tools': bought, 'total_spent_inr': spent, 'total_saved_inr': total_saved, 'rosi_percent':
    round(rosi_percent, 1)}
```

6. How to Run and Test Your Backend (Super Easy!)

Step 1: Start your server:

In your terminal, run: `uvicorn main:app --reload --port 8000`

Step 2: Open the automatic testing screen:

Open Google Chrome and go to: <http://127.0.0.1:8000/docs>

FastAPI gives you a magical interactive screen! You will see buttons for `/api/dashboard` and `/api/investment/optimize`. You can type numbers, click 'Execute', and see your Python code calculate the live answers right in front of your eyes!

Teacher's Secret Hackathon Trick (The Offline Safety Switch):

Hackathon Wi-Fi often gets slow or disconnects. Always put this in your code:

```
MOCK_MODE = True
```

If the network fails during the live presentation, your server instantly returns pre-calculated sample numbers so your presentation NEVER crashes or gets stuck!

7. Your Easy Step-by-Step Checklist to Build This

- [] Install tools: `pip install fastapi uvicorn pydantic pandas scipy`
- [] Create `main.py` and add CORS so the website can talk to your Python server.
- [] Add the `/api/dashboard` route to return the overall risk score and loss.
- [] Add the `/api/investment/optimize` route for the budget slider.
- [] Open <http://127.0.0.1:8000/docs> and click Execute to test your work.
- [] Smile! You have built a rock-solid Python backend for Smart India Hackathon!